Local runtime-driver subprocess controls

Software validation, 2 November 2026. All eight fixed controls passed against the published assembler Driver. Six harmless local Python child processes started and were reaped; the preexisting-signal and expired-deadline controls started no child. The tested assembler source remained unchanged. No sudo, Docker, build, image payload or scientific workload was invoked.

Fixed local software controls. No sudo, Docker or scientific workload.

The driver command prefix was replaced with the current Python interpreter and -c. Successful stdout/stderr and exit0 were retained, and exit7 was reported as a terminal nonzero status. A sleeping child hit its 0.2-second client limit and exited through SIGTERM (-15). A child that installed SIGTERM-ignore reached the same limit and was escalated to SIGKILL (-9); its READY marker confirms the handler had been installed. An injected pending-signal flag also terminated a sleeping child with SIGTERM.

The output control wrote 17 MiB and exited0; the driver rejected the oversized output after that exit. This confirms output-limit detection, not prevention of writes beyond 16 MiB. The observed limit is not a hard disk quota. Both the output bytes and the rejection receipt are retained.

Every started child's receipt records a terminal code and client_unknown=false. Each direct child was absent from /proc when checked after the call. The preexisting pending-signal flag and expired absolute deadline rejected before any subprocess call. Source hashes, exact arguments, PID/PGID receipts and all eight results are in the structured files. No failed controls were discarded or retried.

The prospective software plan fixed all eight cases, a 30-second outer limit and 512 MiB virtual-memory limit. Every sleep command self-expired after five seconds as an additional local bound. These durations are test conditions, not algorithm performance measurements. The complete raw archive preserves 26 files and their per-member hashes.

Coverage boundary. These checks exercise actual user-owned direct children and real signal/wait handling, while earlier tests mocked the assembly workflow. They do not establish sudo signal propagation, root-owned process reaping, descendant/cgroup cleanup, Docker daemon or build termination, or storage-driver behavior. Independent source review and bounded assembly admission remain pending. The 1,260-target cohort, historical Q1 failure and scientific planning limits remain unchanged.